UTN FACULTAD REGIONAL MENDOZA
UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación

ANÁLISIS E IMPLEMENTACIÓN DE UN SISTEMA AUTOMATIZADO DE MONITOREO DE INTEGRIDAD DE ARCHIVOS (FIM) CON EXTRACCIÓN DIFERENCIAL EN ENTORNOS LINUX

Trabajo Final Integrador de la Tecnicatura Universitaria en Programación

Autor: Rodrigo Jesus Noguerol

Mendoza – Argentina

2026

Resumen

El presente trabajo de investigación documenta el diseño, desarrollo e implementación de un Sistema Automatizado de Monitoreo de Integridad de Archivos (File Integrity Monitoring, FIM) altamente escalable, diseñado específicamente para la protección de directorios críticos en entornos operativos Linux. El objetivo central del estudio es superar las limitaciones de las verificaciones tradicionales mediante la integración de un motor nativo en Python que interactúa con la API de *inotify* del kernel a través de la biblioteca *watchdog*.

Esto garantiza una latencia nula en la detección de alteraciones maliciosas. Además del registro criptográfico basado en el algoritmo SHA-256, el sistema incorpora la extracción de evidencia diferencial exacta (Diff) y una orquestación automatizada orientada a reducir drásticamente el Tiempo Medio de Detección (MTTD) ante posibles compromisos o alteraciones de infraestructura crítica.

CAPÍTULO 1: Introducción.....4
1.1 Contexto del problema.....4
1.2 Planteamiento del problema.....4
1.3 Justificación y relevancia del estudio.....4
1.4 Objetivos de la investigación.....5
1.4.1 Objetivo general.....5
1.4.2 Objetivos específicos.....5
1.5 Alcance y limitaciones.....5
CAPÍTULO 2: Marco Teórico.....6
2.1 Seguridad de la Información y la Triada CIA.....6
2.2 Funciones Hash Criptográficas (SHA-256).....6
2.3 Monitoreo de Integridad de Archivos (FIM) y el Subsistema inotify.....6
CAPÍTULO 3: Marco Metodológico.....7
3.1 Tipo de investigación.....7
3.2 Diseño de investigación.....7
3.3 Técnicas de recolección de datos.....7
3.4 Instrumentos.....8
CAPÍTULO 4: Desarrollo de la investigación.....9
4.1 Arquitectura del sistema FIM analizado.....9
4.2 Herramientas utilizadas.....9
4.3 Procedimiento experimental.....9
CAPÍTULO 5: Resultados.....11
CAPÍTULO 6: Conclusiones.....12
CAPÍTULO 7: Recomendaciones.....13
CAPÍTULO 8: Referencias.....14
CAPÍTULO 9: Anexos.....15
Anexo A - Evidencia de Ejecución del Motor FIM.....15
Anexo B - Registro en Base de Datos.....15
Anexo C - Código Fuente: Módulo de Baseline (generar_baseline.py).....16
Anexo D - Código Fuente: Motor de Monitoreo en Tiempo Real (monitor_fim.py).....17

CAPÍTULO 1: Introducción

1.1 Contexto del problema

En la actualidad, las organizaciones dependen de infraestructuras informáticas complejas para gestionar, procesar y almacenar información crítica para su funcionamiento operativo y comercial. Dentro de estas arquitecturas, la integridad de los datos se erige como un pilar fundamental de la seguridad de la información. Los archivos críticos del sistema operativo, los ficheros de configuración de servicios web y los binarios ejecutables constituyen el objetivo principal de atacantes y amenazas persistentes avanzadas (APT). Estos actores maliciosos buscan alterar de forma silenciosa dichas configuraciones para instalar puertas traseras, ejecutar ransomware o establecer mecanismos de persistencia dentro de la infraestructura tecnológica.

1.2 Planteamiento del problema

A pesar de la disponibilidad de soluciones de seguridad en el mercado, una gran cantidad de pequeñas y medianas organizaciones carecen de procesos sistemáticos de auditoría de seguridad que permitan identificar vulnerabilidades e intrusiones en sus sistemas. Históricamente, el monitoreo de integridad ha sido abordado por herramientas de código abierto como Tripwire o AIDE, las cuales basan su funcionamiento en el escaneo periódico del sistema de archivos. El problema fundamental radica en que esta metodología de detección pasiva (escaneos manuales o mediante tareas programadas `cron`) deja extensas ventanas de exposición entre cada evento de verificación. Durante estos lapsos, las alteraciones pasan desapercibidas. Además, cuando se detecta un cambio, las herramientas convencionales notifican la alteración del estado, pero delegan en el administrador la tarea manual de investigar qué líneas de código fueron inyectadas, complicando el análisis forense posterior al incidente. Frente a este escenario, surge la siguiente pregunta de investigación que guía este proyecto: *¿De qué manera puede desarrollarse un sistema automatizado que no solo detecte alteraciones de archivos en tiempo real en entornos Linux, sino que también extraiga y clasifique la evidencia diferencial de forma inmediata para optimizar la respuesta ante incidentes?*.

1.3 Justificación y relevancia del estudio

La identificación temprana de vulnerabilidades y alteraciones permite reducir los riesgos de intrusión, proteger la información sensible y mejorar drásticamente las políticas de respuesta a incidentes. La presente investigación se justifica técnica y operativamente por la necesidad de democratizar el acceso a herramientas de seguridad reactivas utilizando exclusivamente tecnologías de código abierto, como Python, PostgreSQL y, en etapas futuras, plataformas de automatización como n8n.

Desde la perspectiva forense, la implementación de un submódulo de extracción de diferencias (Diff) integrado en el motor FIM representa un valor añadido crítico. Esta característica resuelve la falta de contexto de las alertas tradicionales, reduciendo los tiempos de análisis al entregar al equipo de seguridad (Blue Team) la evidencia exacta del texto modificado en el mismo instante en que se perpetra el ataque.

1.4 Objetivos de la investigación

1.4.1 Objetivo general

Desarrollar, implementar y validar un sistema automatizado de Monitoreo de Integridad de Archivos (FIM) capaz de detectar, registrar y evidenciar modificaciones no autorizadas en directorios críticos de un entorno Linux en tiempo real.

1.4.2 Objetivos específicos

- Configurar una base de datos relacional robusta (PostgreSQL) para centralizar el almacenamiento inmutable de la baseline de integridad y los registros históricos de auditoría.
- Desarrollar un escáner inicial estático en Python para generar el "Estado Cero" del sistema, calculando las firmas digitales SHA-256 y extrayendo metadatos extendidos (propietario y permisos).
- Implementar un motor de monitoreo reactivo de baja latencia utilizando la biblioteca *`watchdog`* para interactuar de forma eficiente con la API del kernel (*`inotify`*).
- Desarrollar un algoritmo de análisis diferencial para comparar el estado en memoria de los archivos e inyectar la evidencia exacta (Diff) en la base de datos ante eventos de modificación.

1.5 Alcance y limitaciones

El presente proyecto abarca el diseño y validación de un motor FIM funcional evaluado en un entorno de laboratorio controlado bajo el sistema operativo Linux (distribución Ubuntu). El monitoreo se delimita a la vigilancia de directorios críticos estándares como:

- **/etc**: Repositorio de archivos de configuración esenciales del sistema.
- **/bin y /usr/bin**: Directorios de binarios ejecutables críticos.
- **/var/www**: Directorios de despliegue de aplicaciones web expuestas. Entre las limitaciones técnicas se asume la dependencia exclusiva de sistemas basados en el kernel de Linux debido a la utilización del subsistema *`inotify`*, y la necesidad de ejecutar el motor de monitoreo con permisos de superusuario (root) para garantizar la lectura de directorios protegidos.

CAPÍTULO 2: Marco Teórico

Este capítulo detalla los fundamentos conceptuales que sustentan la arquitectura del sistema de monitoreo propuesto.

2.1 Seguridad de la Información y la Triada CIA

La seguridad de la información engloba las prácticas y tecnologías destinadas a proteger los activos digitales contra accesos o alteraciones no autorizadas. Este paradigma se rige por la Triada CIA: Confidencialidad, Integridad y Disponibilidad. En el contexto de este trabajo, el enfoque recae sobre la **Integridad**, la cual asegura que la información no sea modificada de forma ilegítima, siendo cualquier alteración detectable y atribuible.

2.2 Funciones Hash Criptográficas (SHA-256)

Una función hash criptográfica es un algoritmo determinista que transforma datos de longitud arbitraria en una salida de longitud fija (resumen o digest). Para este sistema FIM, se emplea el algoritmo SHA-256 (perteneciente a la familia SHA-2), el cual genera un hash de 256 bits. Se seleccionó por su alta resistencia a colisiones y su pronunciado "efecto avalancha", propiedad matemática que garantiza que la alteración de un único bit en el archivo original resulte en una huella digital completamente distinta.

2.3 Monitoreo de Integridad de Archivos (FIM) y el Subsistema inotify

El File Integrity Monitoring (FIM) combina el monitoreo de eventos del sistema de archivos con el cálculo criptográfico para asegurar la inmutabilidad de los datos. A diferencia de los métodos de escaneo por lotes, los sistemas operativos Linux modernos incorporan el subsistema *`inotify`* a nivel del kernel. *`inotify`* se integra con el Virtual File System (VFS) y emite notificaciones asíncronas inmediatas ante operaciones de entrada/salida (creación, modificación, eliminación). Para interactuar con esta API desde el espacio de usuario, el presente proyecto utiliza *`watchdog`*, una biblioteca de alto nivel en Python que abstrae la complejidad de los eventos del kernel y permite construir manejadores de respuesta en tiempo real.

CAPÍTULO 3: Marco Metodológico

Este capítulo describe la metodología científica y técnica empleada para la construcción y validación del sistema FIM, garantizando el rigor académico del proyecto.

3.1 Tipo de investigación

La presente investigación se clasifica como descriptiva y aplicada. Es descriptiva porque detalla exhaustivamente el comportamiento del sistema de archivos Linux y los métodos de alteración de binarios. Es aplicada porque no se limita al análisis teórico, sino que busca construir una solución tecnológica práctica y funcional orientada a resolver el problema real de la falta de visibilidad ante modificaciones no autorizadas en servidores.

3.2 Diseño de investigación

Se adopta un diseño experimental aplicado en un entorno de laboratorio controlado. El método de trabajo seleccionado es iterativo e incremental, estructurando el desarrollo en fases comprobables:

1. Diseño del esquema de persistencia.
2. Desarrollo del cálculo de la línea base (Estado Cero).
3. Implementación del motor reactivo en tiempo real. Este diseño permite aislar el sistema operativo anfitrión (Windows) utilizando el Subsistema de Windows para Linux (WSL), garantizando que las pruebas de inyección de código malicioso no comprometan infraestructura real.

3.3 Técnicas de recolección de datos

Al tratarse de un desarrollo de software de seguridad, la recolección de datos se basa en el registro automatizado de eventos y el análisis de métricas de rendimiento. Las técnicas principales incluyen:

- **Simulación de inyecciones (Ataques controlados):** Modificación intencional de archivos dentro de los directorios monitoreados para evaluar la capacidad de respuesta del sensor.
- **Medición de latencia:** Evaluación del Tiempo Medio de Detección (MTTD), midiendo los milisegundos transcurridos entre la escritura física del archivo y el registro de la alerta en la base de datos.
- **Extracción diferencial:** Captura de las líneas de texto exactas que fueron alteradas (Diff) para componer la evidencia del registro forense.

3.4 Instrumentos

Para la ejecución de la metodología, se definieron los siguientes instrumentos de hardware y software:

- **Entorno de ejecución:** Subsistema de Windows para Linux (WSL) ejecutando la distribución Ubuntu.
- **Lenguaje de programación:** Python 3.x, elegido por su versatilidad y el soporte nativo de bibliotecas criptográficas (*hashlib*).
- **Sensor de eventos:** Biblioteca *watchdog* de Python, que actúa como capa de abstracción sobre el subsistema *inotify* del kernel de Linux.
- **Persistencia:** Motor de base de datos relacional PostgreSQL, utilizado para centralizar y proteger la base de referencia (Baseline) y el log de auditoría.

CAPÍTULO 4: Desarrollo de la investigación.

Este capítulo documenta la implementación técnica inicial del sistema, abarcando desde la configuración de la persistencia hasta la ejecución del monitoreo.

4.1 Arquitectura del sistema FIM analizado

La arquitectura propuesta se basa en un modelo modular. Se dividió la lógica del sistema en dos componentes principales para asegurar la mantenibilidad del código:

- **Módulo de Baseline (*generar_baseline.py*):** Un script encargado de recorrer los directorios críticos asignados, calcular el hash SHA-256 inicial de cada fichero, extraer sus permisos en formato octal, e insertar estos registros como un "Estado Cero" seguro dentro de PostgreSQL.
- **Módulo de Monitoreo (*monitor_fim.py*):** El motor principal que permanece residente en memoria. Instancia un observador (*Observer*) que escucha eventos del sistema de archivos. Al detectar un evento de modificación, compara el estado del archivo en memoria, extrae el diferencial de texto y persiste la alerta.

4.2 Herramientas utilizadas

Para el desarrollo y validación de esta primera etapa del motor FIM, se utilizaron las siguientes herramientas de software: el lenguaje de programación Python (versión 3.x) junto con sus bibliotecas nativas (hashlib, os) y externas (watchdog, psycopg2); el motor de base de datos relacional PostgreSQL; y el editor de código Visual Studio Code. Todo el entorno fue virtualizado y ejecutado sobre el Subsistema de Windows para Linux (WSL) con la distribución Ubuntu.

4.3 Procedimiento experimental

Para la validación de esta primera etapa, se ejecutó el siguiente procedimiento en el entorno de pruebas:

1. Inicialización del servicio PostgreSQL y creación del esquema relacional *fim_db*.
2. Ejecución del escáner de línea base sobre un directorio de pruebas (*./directorio_prueba*), confirmando el guardado del estado inicial.
3. Ejecución del motor en tiempo real.
4. Modificación manual de un archivo de configuración simulado (*config_test.txt*), inyectando una cadena de texto a modo de firma de atacante.
5. Verificación de la consola y la base de datos para confirmar la captura inmediata de la alerta y la extracción exitosa del diferencial (Diff).

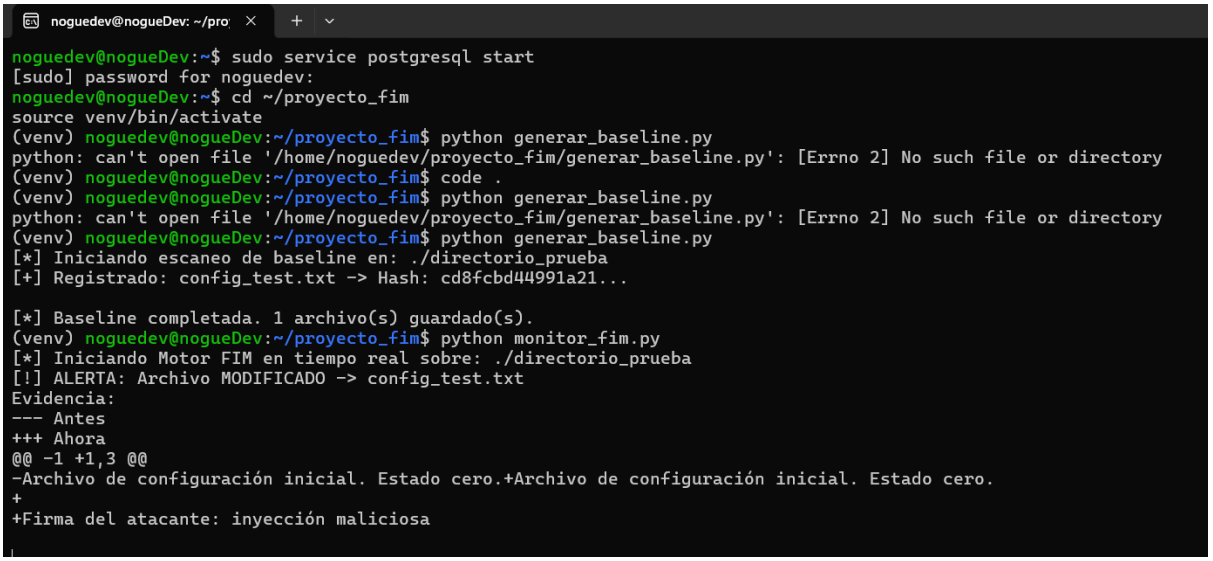


Figura 1: Captura de la terminal del entorno de pruebas (WSL) demostrando la detección en tiempo real de una inyección de código malicioso y la extracción de la evidencia diferencial (Diff).

CAPÍTULO 5: Resultados

Se encuentra en desarrollo y formará parte del informe final de tesis tras concluir la fase de orquestación y respuesta automática.

CAPÍTULO 6: Conclusiones

Se encuentra en desarrollo y formará parte del informe final de tesis tras concluir la fase de orquestación y respuesta automática.

CAPÍTULO 7: Recomendaciones

Se encuentra en desarrollo y formará parte del informe final de tesis tras concluir la fase de orquestación y respuesta automática.

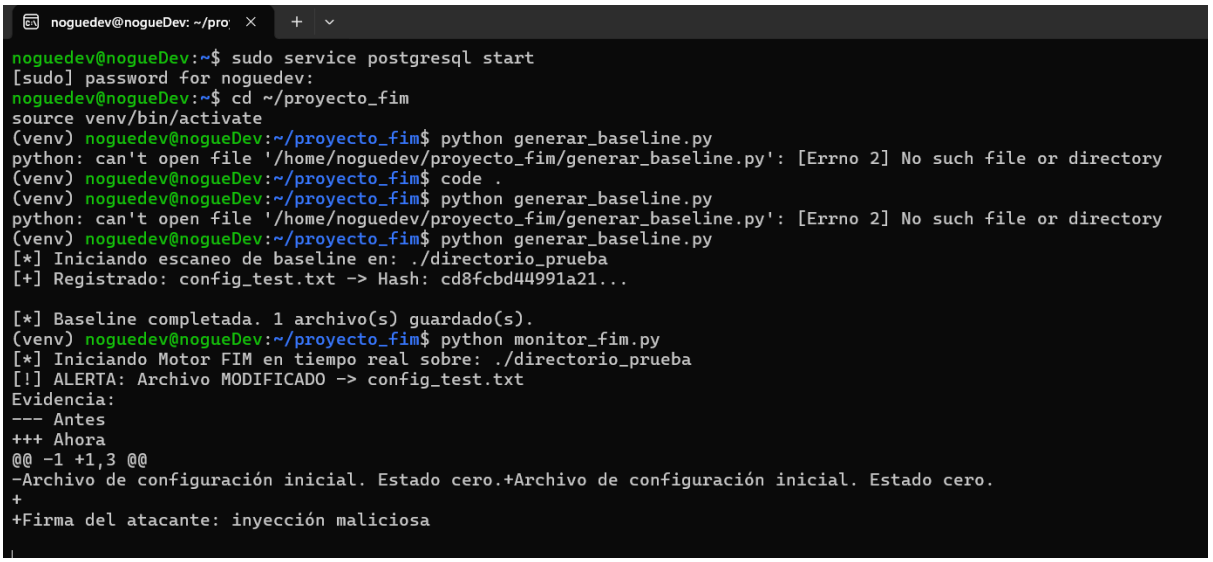
CAPÍTULO 8: Referencias

- Bishop, M. (2003). *Computer security: Art and science*. Addison-Wesley.
- Cortez, A. (2024). *Proyecto 07: Integridad de Archivos (Python)*. Repositorio de GitHub.
- PostgreSQL Global Development Group. (2024). *PostgreSQL Documentation*. Recuperado de .
- Stallings, W. (2018). *Network Security Essentials*. Pearson.
- Wazuh Inc. (2024). *Wazuh documentation: File Integrity Monitoring*. Recuperado de .

CAPÍTULO 9: Anexos

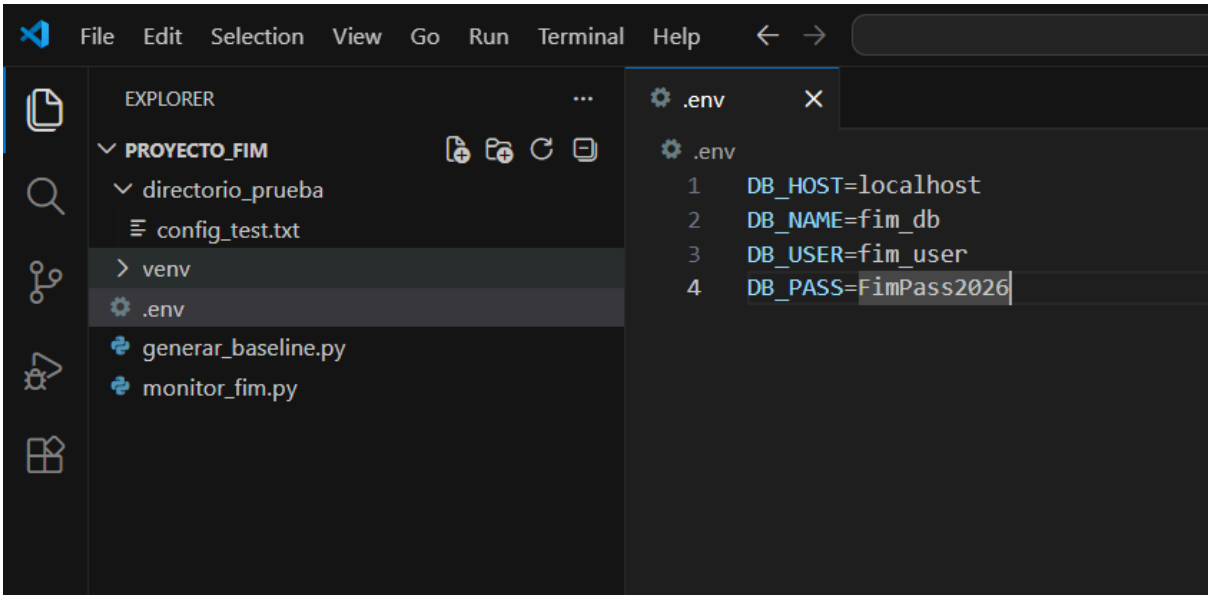
Se encuentra en desarrollo y formará parte del informe final de tesis tras concluir la fase de orquestación y respuesta automática.

Anexo A - Evidencia de Ejecución del Motor FIM



Anexo B - Registro en Base de Datos

Para garantizar la seguridad de las credenciales de la base de datos y evitar la exposición de contraseñas en el código fuente, se implementó un archivo de variables de entorno (*.env*):



Anexo C - Código Fuente: Módulo de Baseline (*generar_baseline.py*)

Este script es el encargado de escanear el directorio crítico, calcular el hash SHA-256 inicial y extraer los metadatos forenses (owner y permisos) para conformar el Estado Cero en la base de datos PostgreSQL.

generar_baseline.py

```python
import os
import hashlib
import psycopg2
from dotenv import load_dotenv

# Cargar variables de entorno del archivo .env
load_dotenv()

DB_HOST = os.getenv("DB_HOST")
DB_NAME = os.getenv("DB_NAME")
DB_USER = os.getenv("DB_USER")
DB_PASS = os.getenv("DB_PASS")

# Directorio de prueba
DIRECTORIO_A_MONITOREAR = "./directorio_prueba"

def get_sha256(filepath):
    """Calcula el hash SHA-256 de un archivo."""
    try:
        sha256_hash = hashlib.sha256()
        with open(filepath, "rb") as f:
            for byte_block in iter(lambda: f.read(4096), b""):
                sha256_hash.update(byte_block)
        return sha256_hash.hexdigest()
    except Exception:
        return None

```

```python
def generar_baseline():
    print(f"[*] Iniciando escaneo de baseline en: {DIRECTORIO_A_MONITOREAR}")

    # Si la carpeta no existe, la creamos y le metemos un archivo
    if not os.path.exists(DIRECTORIO_A_MONITOREAR):
        os.makedirs(DIRECTORIO_A_MONITOREAR)
        with open(os.path.join(DIRECTORIO_A_MONITOREAR, "config_test.txt"), "w") as f:
            f.write("Archivo de configuración inicial. Estado cero.")

    try:
        conn = psycopg2.connect(host=DB_HOST, database=DB_NAME, user=DB_USER, password=DB_PASS)
        cursor = conn.cursor()
        archivos_procesados = 0

        for root, dirs, files in os.walk(DIRECTORIO_A_MONITOREAR):
            for file in files:
                ruta_completa = os.path.join(root, file)
                hash_archivo = get_sha256(ruta_completa)

                if hash_archivo:
                    stats = os.stat(ruta_completa)
                    permisos = oct(stats.st_mode)[-3:]
                    owner = str(stats.st_uid)

                    query = """
                        INSERT INTO registros_archivos
                        (nombre_archivo, ruta_completa, hash_sha256, evento, owner_archivo, permisos)
                        VALUES (%s, %s, %s, 'BASELINE', %s, %s)
                    """
                    cursor.execute(query, (file, ruta_completa, hash_archivo, owner, permisos))
                    archivos_procesados += 1
                    print(f"[+] Registrado: {file} -> Hash: {hash_archivo[:15]}...")

        conn.commit()
        cursor.close()
        conn.close()
        print(f"\n[*] Baseline completada. {archivos_procesados} archivo(s) guardado(s).")

    except Exception as e:
        print(f"[!] Error: {e}")

if __name__ == "__main__":
    generar_baseline()
```

Anexo D - Código Fuente: Motor de Monitoreo en Tiempo Real (*monitor_fim.py*)

Script principal que utiliza la biblioteca watchdog para escuchar eventos del sistema operativo con latencia cero, detectando modificaciones y extrayendo la evidencia diferencial (Diff).

monitor_fim.py

```python
import time
import hashlib
import psycopg2
import difflib
import os
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler
from dotenv import load_dotenv

# Cargar variables de entorno
load_dotenv()

DB_HOST = os.getenv("DB_HOST")
DB_NAME = os.getenv("DB_NAME")
DB_USER = os.getenv("DB_USER")
DB_PASS = os.getenv("DB_PASS")

# Carpeta que vamos a vigilar
PATH_TO_WATCH = "./directorio_prueba"

# Memoria RAM para comparar archivos (el Diff)
memoria_archivos = {}

def leer_archivo_texto(ruta):
    """Lee el contenido de un archivo."""
    try:
        with open(ruta, 'r', encoding='utf-8', errors='ignore') as f:
            return f.readlines()
    except Exception:
        return []

def get_sha256(filepath):
    """Calcula el hash SHA-256."""
    try:
        sha256_hash = hashlib.sha256()
        with open(filepath, "rb") as f:
            for byte_block in iter(lambda: f.read(4096), b""):
                sha256_hash.update(byte_block)
        return sha256_hash.hexdigest()
    except Exception:
        return None
```

```python
def log_to_db(nombre_archivo, ruta_completa, hash_sha256, evento, detalles_diff=""):
    """Guarda el evento y la evidencia en PostgreSQL."""
    try:
        conn = psycopg2.connect(host=DB_HOST, database=DB_NAME, user=DB_USER, password=DB_PASS)
        cursor = conn.cursor()

        # Obtenemos owner y permisos
        try:
            stats = os.stat(ruta_completa)
            permisos = oct(stats.st_mode)[-3:]
            owner = str(stats.st_uid)
        except:
            permisos, owner = "N/A", "N/A"

        query = """
            INSERT INTO registros_archivos
            (nombre_archivo, ruta_completa, hash_sha256, evento, detalles_diff, owner_archivo, permisos)
            VALUES (%s, %s, %s, %s, %s, %s, %s)
        """
        cursor.execute(query, (nombre_archivo, ruta_completa, hash_sha256, evento, detalles_diff, owner, permisos))
        conn.commit()
        cursor.close()
        conn.close()
    except Exception as e:
        print(f"Error en BD: {e}")

class FIMEventHandler(FileSystemEventHandler):
    """Manejador de los eventos de Watchdog"""

    def on_modified(self, event):
        if not event.is_directory:
            ruta = event.src_path
            nombre = ruta.split('/')[-1]
            h = get_sha256(ruta)

            lineas_nuevas = leer_archivo_texto(ruta)
            lineas_viejas = memoria_archivos.get(ruta, [])

            # Sacar el Diff exacto (la evidencia)
            texto_diff = ""
            if lineas_viejas:
                diff_gen = difflib.unified_diff(lineas_viejas, lineas_nuevas, fromfile='Antes', tofile='Ahora')
                texto_diff = ''.join(list(diff_gen))
            else:
                texto_diff = "Contenido modificado. (Sin registro previo en memoria)."

            memoria_archivos[ruta] = lineas_nuevas

            # Guardamos en BD si hubo cambios reales
            if texto_diff:
                log_to_db(nombre, ruta, h, 'MODIFICADO', texto_diff)
                print(f"[!] ALERTA: Archivo MODIFICADO -> {nombre}")
                print(f"Evidencia:\n{texto_diff}\n")

if __name__ == "__main__":
    # Pre-cargar memoria con el estado inicial
    for root, dirs, files in os.walk(PATH_TO_WATCH):
        for file in files:
            ruta = os.path.join(root, file)
            memoria_archivos[ruta] = leer_archivo_texto(ruta)

    print(f"[*] Iniciando Motor FIM en tiempo real sobre: {PATH_TO_WATCH}")
    event_handler = FIMEventHandler()
    observer = Observer()
    observer.schedule(event_handler, PATH_TO_WATCH, recursive=True)
    observer.start()
    try:
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        observer.stop()
        print("\n[*] Motor FIM detenido.")
    observer.join()
```